Applications & Paradigms

Part 2

Anandha Gopalan
(with thanks to Teo Yong Meng)

axgopala@nus.edu.sg

NUS National University of Singapore

Outline

MapReduce Programming Model

Function-as-a-Service

MapReduce Programming Model

Supports arbitrarily divisible workload

Supports distributed computing on large data sets on multiple machines (clusters, public or private clouds, ...)

How large is the data?

Web-scale data on the order of 100s of GBs to TBs to PBs

Input data will not likely fit on a single computer's hard drive

Needs a distributed file system, e.g. Google File System

Inspired by the map and reduce functions in functional programming languages

NUS National University of Singapore

MapReduce Programming Model

Key Idea!

Split data into blocks and assign each block to an instance/process for parallel execution

Merge partial results produced by individual instances after all instances have completed execution

Transforms a set of *input* <**key, value**> **pairs** into a set of *output* <**key, value**> **pairs**

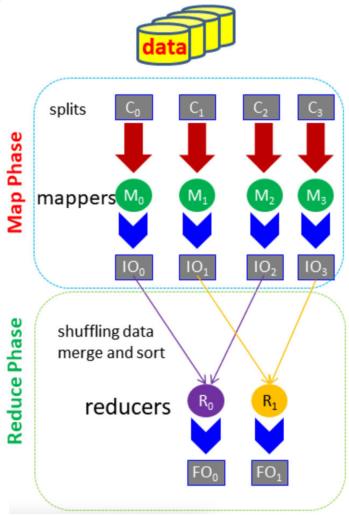
SPMD (Same Program Multiple Data), where a master instance partitions the data and gathers the partial results like GPUs

Programme Structure

1. Read (a lot of) data
2. **MAP** (extract data you need from each record)
3. *Shuffle* and *Sort* data
4. **REDUCE** (aggregate, summarise, filter, transform extracted data, etc.)
5. Write the results

imagine large scale operations on data
e.g. Google search

as a programmer
you only care about map and reduce
everything else is done internally!



you can have multiple layers of reducers
e.g. convergence for ML algos

Divided into two phases – *map* and *reduce*

Splits are processed in isolation by tasks called Mappers

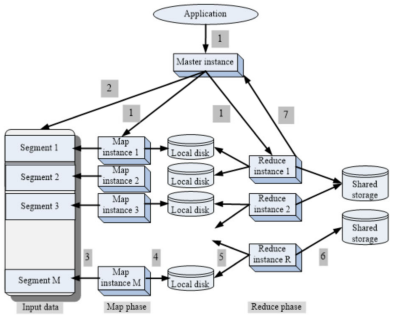
intermediary output stored on local disk

Outputs from mappers, called intermediate outputs (IOs), are brought into a second set of tasks called Reducers

Process of reading intermediate output into a set of reducers is called shuffling

Reducers produce the final outputs (FOs)

who coordinates the steps?
some kind of master node

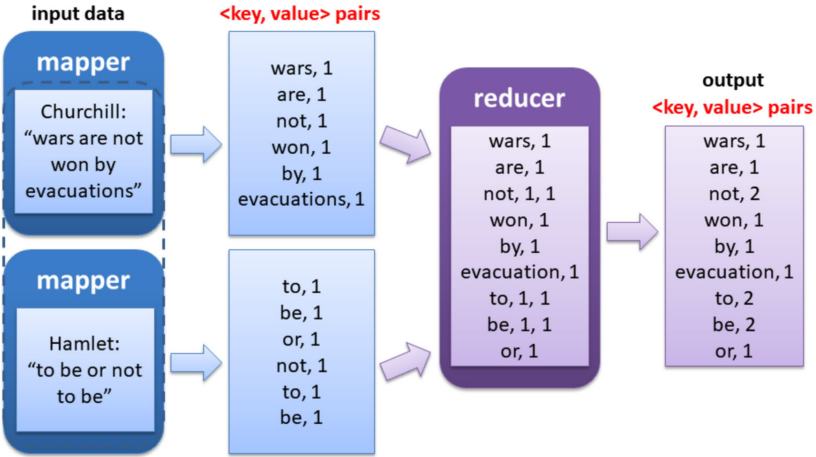


where are all these machines?
the more spread out -> more latency

1. Application starts a master instance, M worker instances for Map phase, and later R worker instances for Reduce phase
2. Master instance partitions input data into M *segments*
3. Each map instance reads its input data segment and processes the data
4. Results of the processing are stored on local disks of the servers where the map instance runs
5. When all map instances have finished processing their data, R reduce instances read the results and **merge** the partial results
6. Final results are written by the reduce instance to a shared storage server
7. Master instance monitors reduce instances and terminates application when all tasks are complete

MapReduce Example – Word Counting

Given a text file, count the number of occurrences of all words in the file



* for simplicity, we left out the sort phase

MapReduce Example – Word Counting

Map – data is partitioned into 4 smaller data files ⇒ split 0 to split 3

Shuffle

- Each smaller data file (split) is evaluated by a map task
- A map produces data (partition) or intermediate results that belongs to different reduce tasks

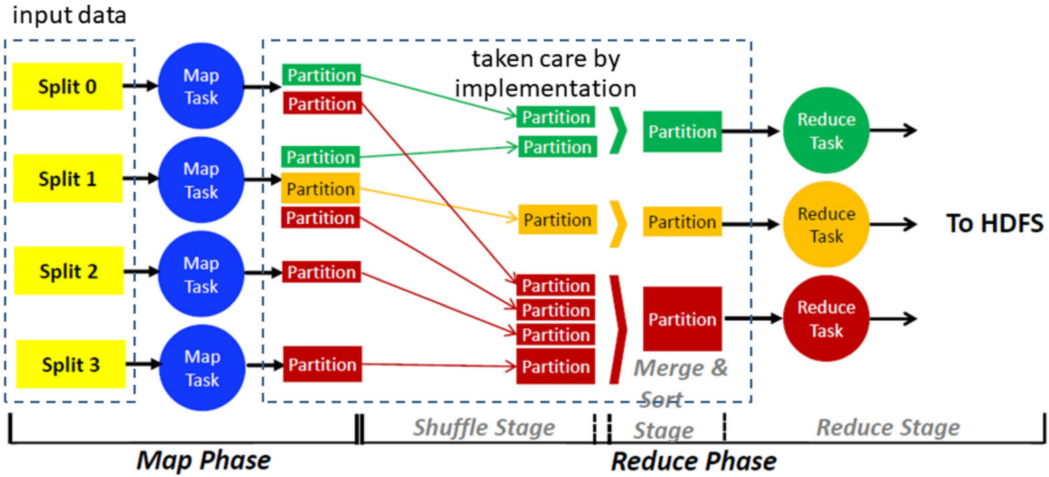
Merge and sort – partitions on arriving at a reduce task are merged into one file and sorted to produce a sorted file

Reduce – evaluates the sorted file to produce the final result

MapReduce programme developer codes only the map and reduce functions ⇒ shuffle, merge and sort are taken care off by the MapReduce system implementation

NUS National University of Singapore

MapReduce Example – Word Counting



MapReduce Example – Word Counting

Map – gets the split data file

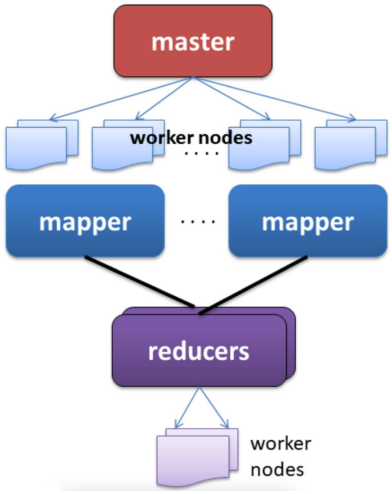
```
function map ()
  tokenize line;
  while (tokenizer has more tokens)
    output <key,value> pair for word
```

token = independent piece of info

Reduce – gets the word along with a list of 1s

```
function reduce ()
  int sum = 0;
  sum = total number of 1s
  output <key,value> pair for word
```

NUS National University of Singapore



- Master program creates data-localised tasks to minimise network communication
  minimal communication -> minimum latency
- Worker nodes switch between mapper and reducer cost reduction: reuse the machine that did mapping to do reducing
- **Map** workers run in parallel
  - Each has its own data set
  - Each creates intermediate local values from each input data set
- **Reduce** workers run in parallel
  - May reduce intermediate data
  - There is an optimal number of reducers for best performance

Google

Indexing for Google search

Organising articles in Google news

Performing various user statistics

Meta

Data Mining

Optimising ads for users

Detecting spam on Facebook

Issue

Google MapReduce – closed source, developed by Google (2004) to process large amounts of raw data

Solution

Hadoop MapReduce – open source, developed by Apache + Yahoo

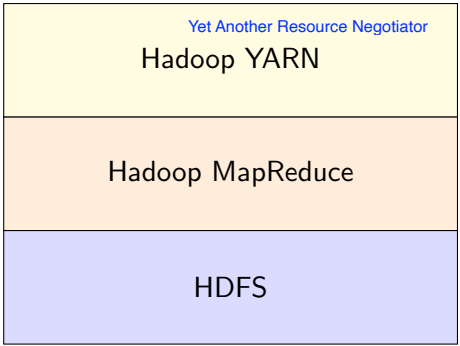
Hadoop

large scale data processing

Presents MapReduce as an analytics engine with a distributed storage layer referred to as Hadoop Distributed File System (*HDFS*)

HDFS mimics Google File System (*GFS*)

Very popular implementation of MapReduce



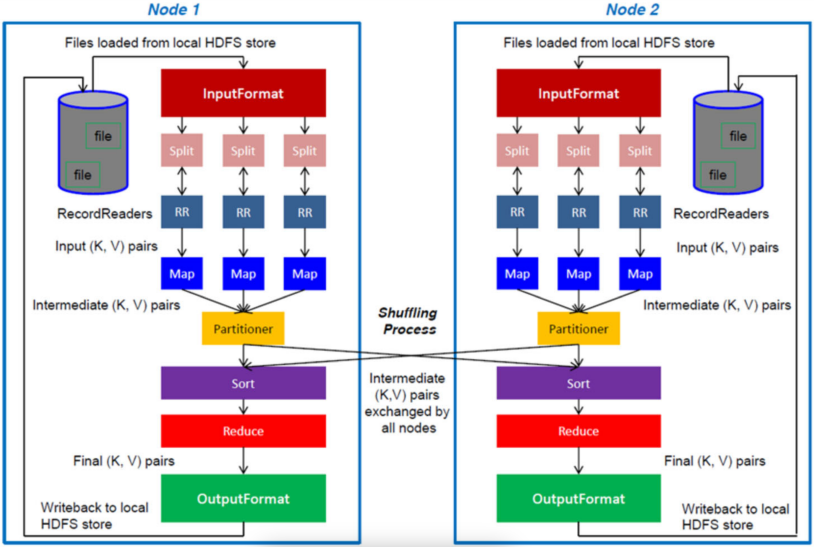
Resource Allocation

Scheduler

Computation

Storage

Hadoop on Multi-Node



where is slow?
why store things on disk?

Apache Spark

because nowaways more memory available than 5 years ago

Fast, **in memory** data processing engine

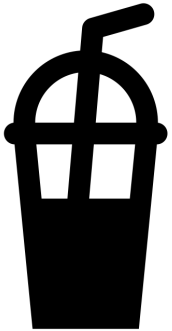
Can run independently or on top of Hadoop

Designed for real-time and advanced analytics

Feature	Apache Spark	Hadoop (MapReduce)
Batch Processing	Yes speedup	Yes
Real-time Processing	Yes	No
Machine Learning	Built-in (MLlib)	Need external tools

NUS National University of Singapore

Take a Break







NUS National University of Singapore

Function-as-a-Service (FaaS)

Approach to build scalable, reactive, event-driven applications

there's an event you want to react to it as fast as you can

Uses functions as unit of deployment – application is divided into smaller, single-purpose (**Lambda**) functions

Event triggers a lambda function and exits after execution

No idle capacity!

e.g. fire alarm -> you just react and run

Lambda is stateless

just like REST API!

Serverless FaaS

server is taken care of for you

Build (serverless) applications without the need to provision or manage servers ⇒ consumer only pays when code is executing

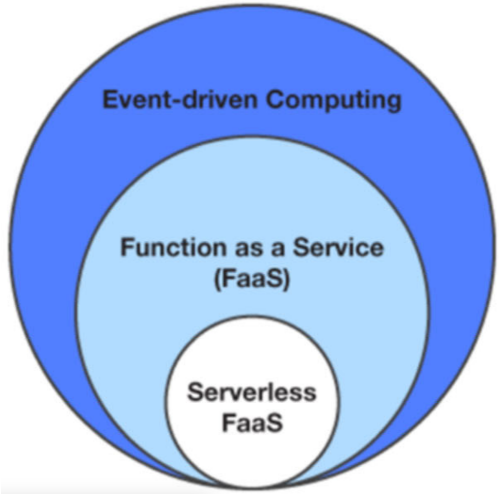
No VMs/containers in the programming model

Vendor provides provision-free scalability, i.e., scale with workload

Use Cases

Web/mobile backends, real-time data processing, chatbots, virtual assistants, ...

FaaS


Event-driven Computing
Function as a Service
(FaaS)
Serverless
FaaS

Functions should perform only one action lightweight

Functions should be self-contained $\Rightarrow$ should not call other functions

Use as few libraries as possible

FaaS vs PaaS vs Containers vs VMs I

Provisioning time is much lower – ms vs minutes vs hours!

Ongoing administration is almost none

Allows for elastic scaling as each action is inherently scaled

No capacity planning required

Limited ability to persist connections and state must be kept in an external service or resource disadvantage!

stateless
just like RESTApi
cookies -> maintain state

NUS National University of Singapore

FaaS vs PaaS vs Containers vs VMs II

FaaS provider manages all maintenance (similar to PaaS)

High availability is inherent in the FaaS model

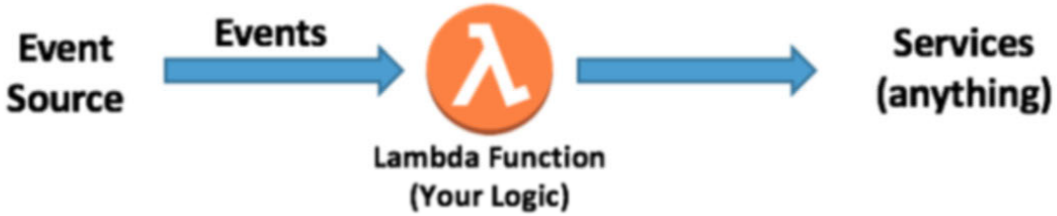
Resource utilisation is excellent

another advantage: easier debugging
cos functions are smaller

FaaS model has resource limits

NUS National University of Singapore

How a Lambda Function Runs



How a Lambda Function Runs

Lambda function – your application logic, i.e., code you want to execute (Java, Node.js, Python, ...)

Configuration – defines how your code is executed, i.e., configure event sources that invoke your functions

Example

Processing an event when an API gateway receives an HTTPS request

System takes care of code to integrate event source with lambda function, manage infrastructure that detects events and invoke lambda, etc.

NUS National University of Singapore

3 Steps

1. **Create lambda service**
   - Business logic (code)
   - Configuration – region to run, AWS credentials, etc.
   - Event – the event that triggers a lambda function
2. **Deploy service created**
3. **Invoke/test your service**

Using AWS Lambda

Web Interface

AWS Management Console
(`https://aws.amazon.com/lambda/getting-started/`)

Command Line Interface

AWS SAM CLI (`https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless-getting-started-hello-world.html`) Serverless Application Model

There are also some 3rd party CLIs available

NUS National University of Singapore

Next Week

More Examples! ☺

References

What is Function as a Service (FaaS)?
(https://www.ibm.com/think/topics/faas)

MapReduce: Simplified Data Processing on Large Clusters
(https://static.googleusercontent.com/media/research.google.com/en/
/archive/mapreduce-osdi04.pdf)

Apache Hadoop (https://hadoop.apache.org/)

The Hadoop Distributed File System
(https://ieeexplore.ieee.org/document/5496972)

NUS National University of Singapore

Continuous Feedback



https://forms.cloud.microsoft/r/ZJxTf2wFjr

NUS National University of Singapore